ACCRA TECHNICAL UNIVERSITY

ATU VIRTUAL INTERNSHIP

SOFTWARE DEVELOPMENT

STUDENT STUDY PLANNER WEB APPLICATION

INDIVIDUAL MINI PROJECT REPORT

Student Name and ID: GRANT OWUSU ASAMOAH - 01251069C

Programme: Diploma in Information Technology

Institution: Accra Technical University

Internship: ATU Virtual Internship

Project Type: Individual Mini Project

Submission Date: 1 November 2026

Project Title: Student Study Planner Web Application

Contents

1 Problem Statement **2**

2 Objectives **2**

3 Technologies Used **2**

3.1 HTML5 2

3.2 CSS3 2

3.3 JavaScript 3

3.4 Browser Local Storage 3

3.5 GitHub 3

3.6 Deployment Platform 3

4 Features Implemented **3**

5 Interface Design **4**

5.1 Screenshot of the Working Application 4

6 Testing and Results **5**

6.1 Additional Tests 6

6.2 Testing Summary 6

7 Challenges and Reflection **7**

8 Deployment and Source Code **7**

9 AI Assistance Declaration **7**

1. Problem Statement

Students often juggle assignments, revision, and reading across many courses and may lose track of completed and pending academic activities.

A simple study planner can help students record tasks by subject, monitor their progress, and remove tasks that are no longer needed.

The intended users of this application are students who want to organise their academic work efficiently. The Student Study Planner Web Application provides a simple and accessible way to manage study tasks in one place.

2. Objectives

The main objective of this project is to develop a simple and responsive web application that helps students manage their study activities.

The specific objectives are:

1. To allow students to add study tasks with a task title and subject.
2. To allow users to mark tasks as completed or incomplete.
3. To enable users to delete tasks that are no longer required.
4. To save study tasks in the browser so that they remain available between visits.
5. To display task summaries showing the total, completed, and pending tasks.
6. To provide a responsive and accessible user interface for different screen sizes.

3. Technologies Used

The following technologies and tools were used in the development of the Student Study Planner Web Application.

3.1 HTML5

HTML5 was used to structure the web page. It provides the header, summary cards, task input form, task list, and footer.

3.2 CSS3

CSS3 was used to style the application with blue, white, and light-grey colours. It also provides cards, badges, responsive layouts, and visual styling for completed tasks.

3.3 JavaScript

Vanilla JavaScript was used to implement the application's functionality, including adding, validating, completing, deleting, and displaying tasks.

3.4 Browser Local Storage

Browser Local Storage was used to save tasks so that they remain available after the page is refreshed or reopened in the same browser.

The data is stored locally in the browser and does not automatically synchronise across different devices.

3.5 GitHub

GitHub was used for version control and source-code hosting.

3.6 Deployment Platform

The application was intended to be hosted online using a free hosting platform.

Platform used: [KEEP ONLY THE PLATFORM ACTUALLY USED: GitHub Pages or Netlify]

4. Features Implemented

The application includes the following features.

1. **Add Task:** Users can add a task by entering its title and subject.
2. **Display Tasks:** All saved tasks are displayed in the task list.
3. **Mark Complete:** Users can mark a task as completed.
4. **Mark Incomplete:** Users can return a completed task to the pending state.
5. **Delete Task:** Users can remove individual tasks from the list.
6. **Local Storage:** Tasks are saved in browser Local Storage and loaded when the application is opened again.
7. **Input Validation:** The application checks user input and displays visible error messages when the task title is missing or contains only spaces.
8. **Empty-State Message:** A message is displayed when there are no tasks in the list.
9. **Summary Counters:** The application displays the total number of tasks, completed tasks, and pending tasks.

10. **Responsive Layout:** The interface adapts to different screen sizes.
11. **Accessibility:** Labels, visible focus outlines, semantic HTML, and text-based status indicators help make the application accessible.
12. **Safe Text Display:** User-entered text is inserted using JavaScript's `textContent` property so that it is displayed as text rather than interpreted as HTML.
13. **Task Priority:** As an optional extra, users can assign Low, Medium, or High priority to tasks. Priority badges are displayed and saved in Local Storage.

5. Interface Design

The application interface is designed to be simple, clear, and easy to navigate.

The main interface consists of the following components:

- **Header:** Displays the application title, subtitle, and a "Saved locally" indicator.
- **Summary Cards:** Three cards display the total, completed, and pending tasks.
- **Add Task Form:** Allows users to enter task details and select a priority.
- **Task List:** Displays each task with a checkbox, subject, status badge, priority badge, and Delete button.
- **Footer:** Appears at the bottom of the application.

5.1 Screenshot of the Working Application

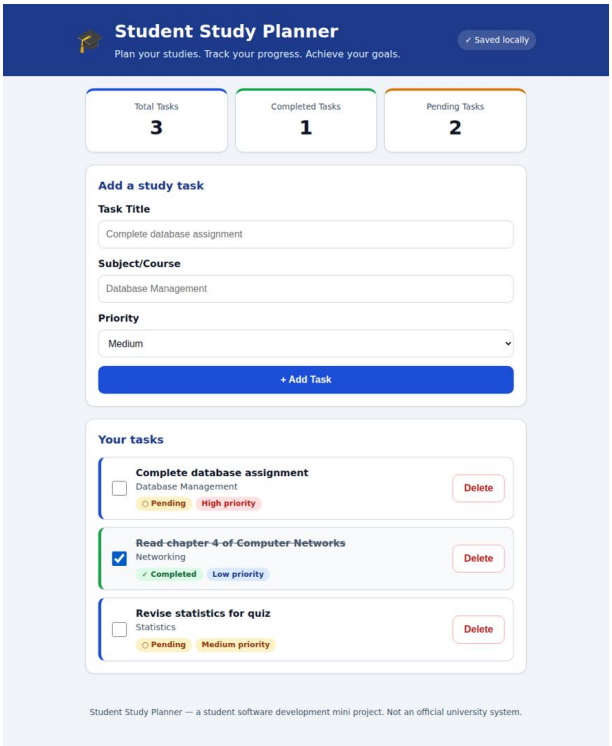


Figure 1: User interface of the Student Study Planner Web Application.

6. Testing and Results

Tests were performed automatically using headless Chromium and Playwright against the final code by opening `index.html` as a local file.

The expected and actual results were compared to verify that the application's main functions worked correctly.

Note: Additional manual testing should be carried out in a browser or on a mobile phone and on the deployed application URL before submission.

Table 1: Functional Testing Results

No.	Test Case	Expected Result	Actual Result	Status
1	Add task with valid title and subject	Task appears in the list	One task shown with title, subject, and priority; form cleared	Pass
2	Submit an empty title	Input rejected with a message	No task added; "Please enter a task title." displayed	Pass
3	Enter a title containing only spaces	Input rejected	No task added; validation message displayed	Pass
4	Mark an incomplete task complete	Task visibly marked complete	Completed styling, badge, and line-through displayed	Pass

No.	Test Case	Expected Result	Actual Result	Status
5	Mark a completed task incomplete	Task returns to normal state	Normal styling and Pending badge restored; no duplicate	Pass
6	Delete a task	Selected task removed	Only the chosen task removed; other tasks retained	Pass
7	Refresh the browser	Tasks remain saved	Three tasks remained; counters showed 3/1/2 and completed state was retained	Pass
8	Delete every task	Empty-state message appears	Message displayed; counters showed 0/0/0	Pass
9	Add a task after deleting all tasks	New task appears and empty message disappears	One task shown; empty-state message hidden	Pass
10	Add several tasks	Each task appears separately	Three tasks displayed with correct titles and subjects	Pass
11	Check summary counters	Counters display correct values	Counters showed 1/1/0 and then 3/1/2	Pass
12	Test a narrow screen of 360 pixels	No horizontal scrolling	Page width was 360 pixels, matching the viewport	Pass

6.1 Additional Tests

Two additional checks were performed.

- **Corrupted Local Storage Data:** Invalid stored data resulted in an empty task list without causing application errors.
- **Safe Text Handling:** Input such as `<img onerror=...>` was displayed as plain text instead of being interpreted as HTML.

6.2 Testing Summary

The recorded automated tests passed, and no defects were reported during these tests. Therefore, no fixes or retests were needed for the tested cases.

Manual browser, mobile-device, and live-deployment checks should also be completed and documented before submission.

7. Challenges and Reflection

Important: Replace this section with your own experience. The following paragraph is only an example and should be used only if it accurately describes your work.

Keeping the summary counters and Local Storage data consistent with the task list was challenging at first. I addressed this by ensuring that each task operation updated the stored data and refreshed the displayed list and counters.

This helped me understand how JavaScript functions can work together to manage application state and update the user interface.

In a future version, I would improve the application by adding due dates, task editing, and more advanced study planning features.

8. Deployment and Source Code

The application should be deployed online so that it can be accessed through a web browser.

- **Live Application:** `https://grant07577.github.io/student-study-planner/`
- **GitHub Repository:** `https://github.com/Grant07577/student-study-planner`

Replace the placeholder links with the actual URLs of the deployed application and source-code repository.

Ensure that both links work before submitting the report.

9. AI Assistance Declaration

An AI assistant, Claude by Anthropic, assisted with planning, generating the HTML, CSS, and JavaScript, running the tests, and drafting this report and README.

I reviewed the code and am responsible for this submission.

Note: Edit this declaration to accurately describe the AI tools used and your own contribution to the project.

Conclusion

The Student Study Planner Web Application was developed to help students organise and manage their academic tasks.

The application provides features for adding tasks, marking tasks as complete or incomplete, deleting tasks, viewing summary counters, and saving data locally in the browser.

The project demonstrates the practical application of HTML5, CSS3, JavaScript, and browser Local Storage in building a responsive web application.

The recorded test results indicate that the tested features performed as expected. Further manual testing and verification of the deployed application should be completed before final submission.